

TIMELINER

System Architecture Specification & System Implementation Prompt

Project Type	Target Stack	Cost Model
Zero-Cost AI News Platform	Flutter / PWA, Supabase, Gemini API	\$0/month (100% Free Tiers)

1. Executive Summary & Core Objective

Timeliner is an autonomous, cross-platform news portal that ingests top news daily (focusing on Indian regional and national coverage) and transforms continuous news items into interactive, chronological event timelines.

The primary constraint of the project is to maintain a **\$0/month operating cost indefinitely** while guaranteeing zero manual maintenance, high availability, instant page load speeds, and platform independence (iOS, Android, Web).

2. System Architecture Blueprint

To maintain zero cost and prevent rate-limiting, the system decouples write/ingestion operations from read queries via a **Batch-Processed Serverless Pipeline**.

3. Free-Tier Stack Configuration Matrix

Component	Selected Technology	Free Tier Limits	Usage Strategy
Compute & Automation	GitHub Actions Workflows	2,000 build mins/month	Runs 3x daily Python cron scripts (~30s runtime = ~45 mins/mo).
AI Engine	Google Gemini 2.5 Flash API	1,500 free requests/day	Batch processing consumes ~60 requests/day (~4% of free limit).
Database	Supabase Postgres	500 MB DB / 50k monthly active	Stores structured JSON nodes. ~15 MB storage footprint per year.
Hosting / CDN	Vercel / Cloudflare Pages	Unlimited bandwidth	Static web application deployment with instant global edge caching.
Push Alerts	Firebase Cloud Messaging	Unlimited free notifications	Cross-platform Web Push (VAPID) and mobile notifications.
News Source	Public RSS Feeds	Unlimited access	Pulls Google News IN RSS & PIB directly without requiring paid APIs.

4. Database Schema (Supabase Postgres DDL)

```
-- 1. Overarching News Topics
CREATE TABLE stories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  category TEXT DEFAULT 'India Top News',
  summary TEXT,
  last_updated TIMESTAMPTZ DEFAULT NOW(),
  is_active BOOLEAN DEFAULT TRUE
);

-- 2. Chronological Timeline Nodes
CREATE TABLE timeline_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  story_id UUID REFERENCES stories(id) ON DELETE CASCADE,
  event_timestamp TIMESTAMPTZ NOT NULL,
  headline TEXT NOT NULL,
  details TEXT,
  source_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Daily News Digest Index
CREATE TABLE daily_digests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  digest_date DATE NOT NULL,
  story_id UUID REFERENCES stories(id) ON DELETE CASCADE,
  summary_snippet TEXT,
  display_order INT DEFAULT 0
);
```

5. Deduplication & AI Prompting Logic

To prevent repeated nodes when updating ongoing stories, the background script performs a lookup before passing context to Gemini.

PROMPT SPECIFICATION Gemini 2.5 Flash Deduplication Prompt:

```
"You are a structured news analyst. Here is the existing chronological timeline for story  
'{story_title}':  
{existing_timeline_json}
```

```
Here are today's new RSS news articles for this topic:  
{new_rss_articles_text}
```

```
TASK: Extract any NEW significant chronological events that occurred today. Output STRICT JSON  
adhering to this schema: array of objects with keys: 'event_timestamp', 'headline', 'details'. If  
there are NO new events, return an empty list []. Do NOT duplicate existing points."
```

6. Frontend Layout & Interactive Mechanics

The "Shift-Left" UI State Architecture

- **State 1 (Default / Feed View):** Main view renders the top stories for the current date in a full-width column. The bottom control bar contains a horizontal carousel of historical dates.
- **State 2 (Story Selection Transition):** When a user taps a news card:
 - The News Feed panel animates leftwards (shrinking from **100% width to 35%** on desktop/tablet, or sliding off-screen on mobile).
 - The Timeline panel slides in from the right (**65% width**), rendering the ordered chronological milestones with connecting vertical lines.
- **State 3 (Historical Date Focus):** Tapping a date in the bottom carousel centers that day and fetches the historical digest (``SELECT * FROM daily_digests WHERE digest_date = "YYYY-MM-DD"``).

7. Copy-Paste AI Prompt for Downstream Development Agents

AGENT PROMPT INSTRUCTION Feed this block to any LLM/Coding Assistant to auto-generate the complete codebase:

"You are a senior full-stack software engineer. Build the 'Timeliner' application using the following strict architectural constraints:

1. STACK REQUIREMENTS:

- Frontend: Flutter (Cross-platform for Web, Android, iOS) or Next.js PWA.
- Database: Supabase Postgres.
- Automation: Python script designed for GitHub Actions running on a cron schedule.
- AI API: Google Gemini 2.5 Flash API with `response\_mime\_type='application/json'`.
- Push Alerts: Firebase Cloud Messaging (FCM) VAPID Web Push.

2. KEY FEATURES:

- Automated ingestion of Indian daily top news RSS feeds.
- AI summarization into structured JSON timelines.
- Deduplication algorithm: check existing story events before calling Gemini to append only NEW timeline milestones.
- UI Layout: Mobile-first responsive design. Implement the 'Shift-Left' UI pattern where selecting a news item shifts the main news list left (35% width) and slides in the timeline view (65% width). Include a bottom date selector for browsing historical news digests.
- Operating Cost Limit: Must run entirely on \$0/month free tier limits.

Please generate the complete, production-ready code structure including the Supabase migrations, Python backend automation script, and Frontend UI components."